

Scheda — FI2 LAB04a: Frazioni base, la libreria FrazLib

Materiale: LAB-04-Insiemi di frazioni.pdf (sl. 4-21), Lab04a-FrazioniBase-Startkit.zip (solo FrazLibTest) · **Modalità:** guidata · **Tempo del docente:** 20 minuti [sl. 21] **Teoria che entra qui:** 08 □ — package e import sl. 11-16, 18-20, default package «innominabile» sl. 35 · 07 □ — array di oggetti sl. 17-19, length sl. 19, *for each* sl. 41-43.

Setup

- dopo `git pull` in `~/cockpit`: *Existing Projects* → *Select root directory* → `esame_FI2/da_importare/LAB04a_FrazioniBase-011026` → spunta **Copy projects into workspace** → **Finish**. È già rinominato, ha la compliance 21 e la configurazione `FrazLibTest` con `-ea`;
- ristruttura** come da sl. 19: package `util` (`MyMath`), frazione (`Frazione`, `FrazioneTest`, `MainFrazione` del **tuo LAB03**), `frazlib` (`FrazLib` da creare, `FrazLibTest` già lì). Copia i file da LAB03 incollandoli sul package: Eclipse riscrive la riga package;
- X rosse attese: `MyMath` cannot be resolved in `Frazione` e `FrazioneTest` (manca un `import`, sl. 18) e `FrazLib` cannot be resolved in `FrazLibTest` (la classe non esiste ancora);
- prima di `FrazLib`, lancia `FrazioneTest` (con `-ea`): deve restare verde dopo lo spostamento.

Ordine di lavoro

#	Cosa	Firma (da FrazLibTest)	Test	Costrutto nuovo → slide
1	packages + import	<code>frazione.Frazione</code> <code>util.MyMath</code>	<code>FrazioneTest</code> verde	package, import — 08 sl. 18-20, LAB sl. 16-18
2	<code>FrazLib.sum</code>	<code>public static Frazione sum(Frazione[] tutte)</code>	due assert su somma	static in un «ente terzo» — LAB sl. 7-9; array di oggetti — 07 sl. 17-19
3	<code>FrazLib.mul</code>	<code>public static Frazione mul(Frazione[] tutte)</code>	due assert su mul	stessa struttura di sum

Casi limite da pensare

- Perché `sum` non è un metodo di `Frazione` e non è un metodo dell'array? (sl. 5-8; Δ è l'errore `persone[].getMediaEta` del 16/09.)
- Perché `FrazLibTest` **non potrebbe** usare `Frazione` se restasse nel default package? (sl. 13, 16.)
- `Frazione` ha già `sum` e `mul` per due frazioni: quanto aritmetica nuova serve in `FrazLib`? (Δ «*un metodo della classe lo fa già?*», confronto LAB03.)
- Da quale valore parte il risultato prima di aver visto la prima frazione? Che cosa dovrebbe restituire `sum` di un array **vuoto**, e `mul`?
- Il secondo caso di `sum` attende 179/420: il risultato è ridotto ai minimi termini da te o da chi?
- Due `assert` per metodo, entrambi con quattro frazioni: chi controlla un array di **una** sola frazione? (Δ pattern 2: test verde $\neq$ metodo corretto.)

Test rosso?

Non compila: package o import sbagliati, classe nel default package · `AssertionError` = failure: la riga indica l'assert · `NullPointerException` = error: una cella dell'array non contiene un oggetto (07 sl. 17) → prontuario §1, oppure scrivilo in chat. Silenzio = verde **solo con -ea**.

Dopo i test verdi

- copia il progetto in `corsi/FI2/esame_FI2/svolti/LAB04a_FrazioniBase/`;
- confronto con la soluzione del docente → `confronto_LAB04a.md`;
- poi LAB04b (`Frazione` «double face», sl. 22-24): la stessa libreria finisce dentro `Frazione`.